Building a Steam Analytics Backend: A MongoDB Case Study

From a 1.5GB Dataset to a Query-Ready Database

The Mission: Powering a Game Discovery & Analysis Platform

The goal is to build the data backend for a comprehensive Steam analytics application. The architecture needed to support three core user-facing features:

Game Browser

Allow users to search and filter over 85,000 games by genre, tags, price range, and user ratings.

Review Analysis

Enable the aggregation and analysis of user reviews to derive sentiment and trends.

Price Tracker

Support the comparison of prices across different regions and the tracking of historical discounts.

The Dataset: A Deep Dive into Steam's Catalogue

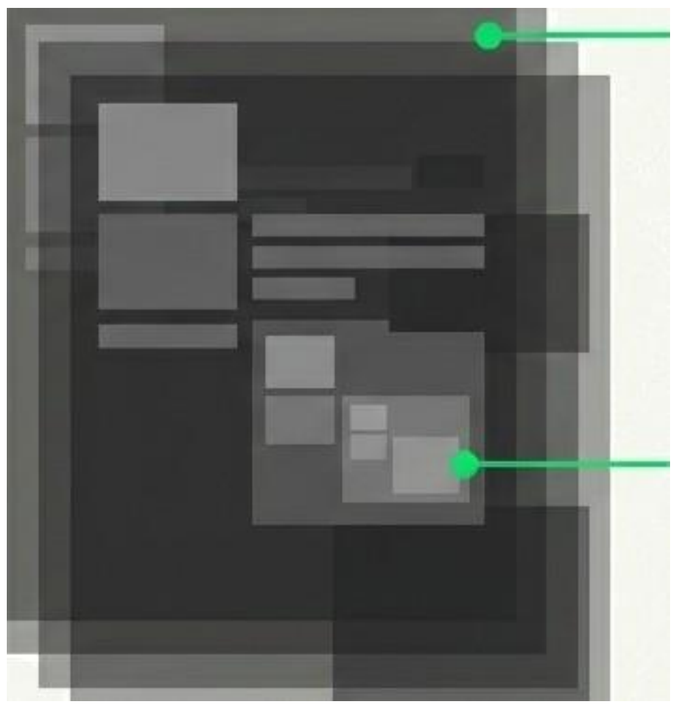
The project is built upon the 'Steam Games Complete Dataset,' a rich and complex collection of public game data.

Key Statistics

Source	haggle (fronkongames/steam—games—dataset)
Size	1.5 GB
Records	Over 85,000 unique games

Content Overview

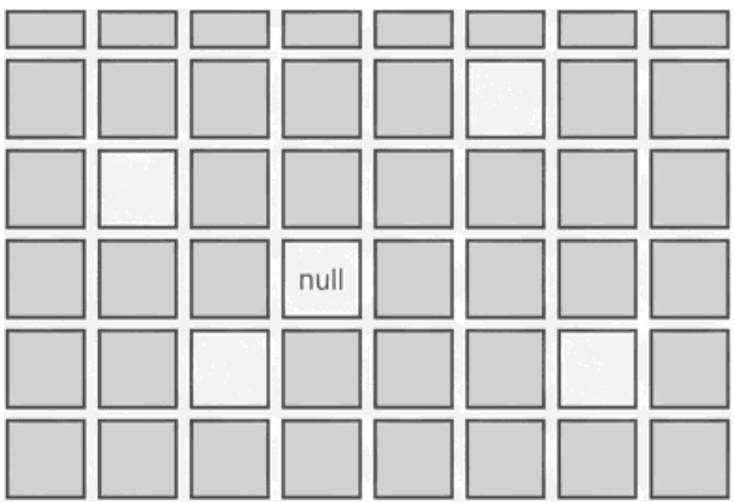
- Core metadata (name, release date, developers)
- User-generated content (reviews, tags, ratings)
- Pricing information across multiple currencies
- Technical details (system requirements, achievements, DLCs)



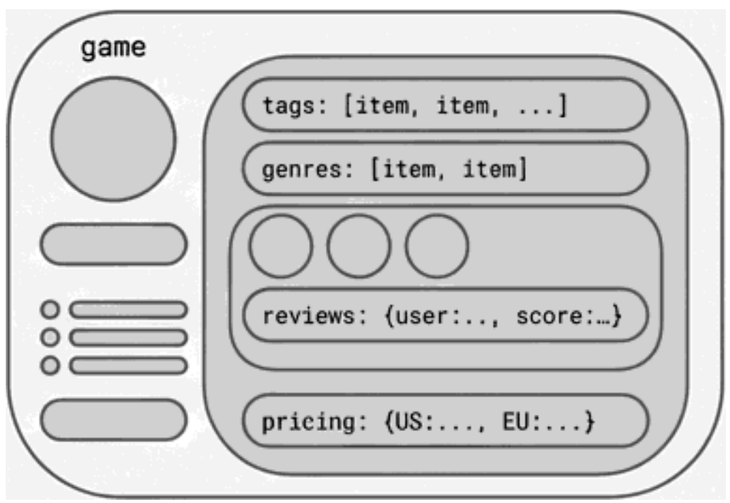
The Challenge: Taming a Complex, Real-World Dataset

The dataset's structure presents significant challenges for traditional relational databases. Its real-world nature means it is not uniform or perfectly clean.

Rigid SQL Table



Flexible Document



Key Structural Hurdles

- **Variable Schema**: Not all games have the same data points. Fields for `achievements`, `DLCs`, or `Metacritic score` are often absent.
- **Nested Arrays**: Core attributes like `tags`, `genres', and `categories are naturally stored as lists of varying length.
- **Embedded Documents**: Complex entities like user `reviews` and regional `pricing` are best represented as nested objects within a single game document.

Data Quality Issues

- Inconsistent data formats (e.g., ”Liser score‘ can be Number or String).
- Missing v&ues are common for ”release dates' and ”price’ (for free games).

The Strategic Choice: Why MongoDB Was the Right Tool

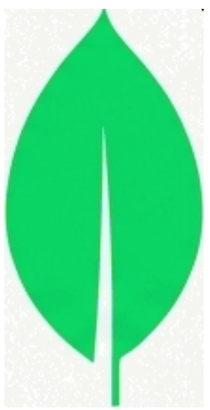
A document-oriented database was selected to embrace the dataset's inherent structure, not fight it. MongoDB provided a clear advantage over a traditional SQL approach.

Flexible Schema

Natively handles documents with varying fields, eliminating the need for sparse tables or complex joins.

Rich Data Structures

Easily stores nested arrays and embedded documents, mirroring the source data's structure.

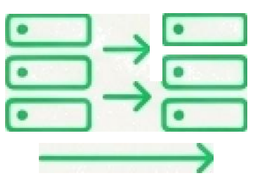


Powerful Querying

Built-in support for querying within arrays and sub-documents is ideal for the application's filtering needs.

Text Search

Native text indexing is essential for searching game descriptions and user reviews.



Scalability

Proven horizontal scaling capabilities provide a future-proof path for handling massive review volumes.

The Blueprint: A Query-First Schema Design

The database schema was designed to optimize for the application's most common read operations. A two-collection model was implemented to balance detail and performance.

James collection

The primary collection, containing one document per game.

Embeds ‘tags’, genres’, categories* as arrays for rast filtering.

Nests achievements', DLCs', and “pricing as sub-documents.

Designed for quick retrieval of a game's complete profile.

‘app id’

reviews collection

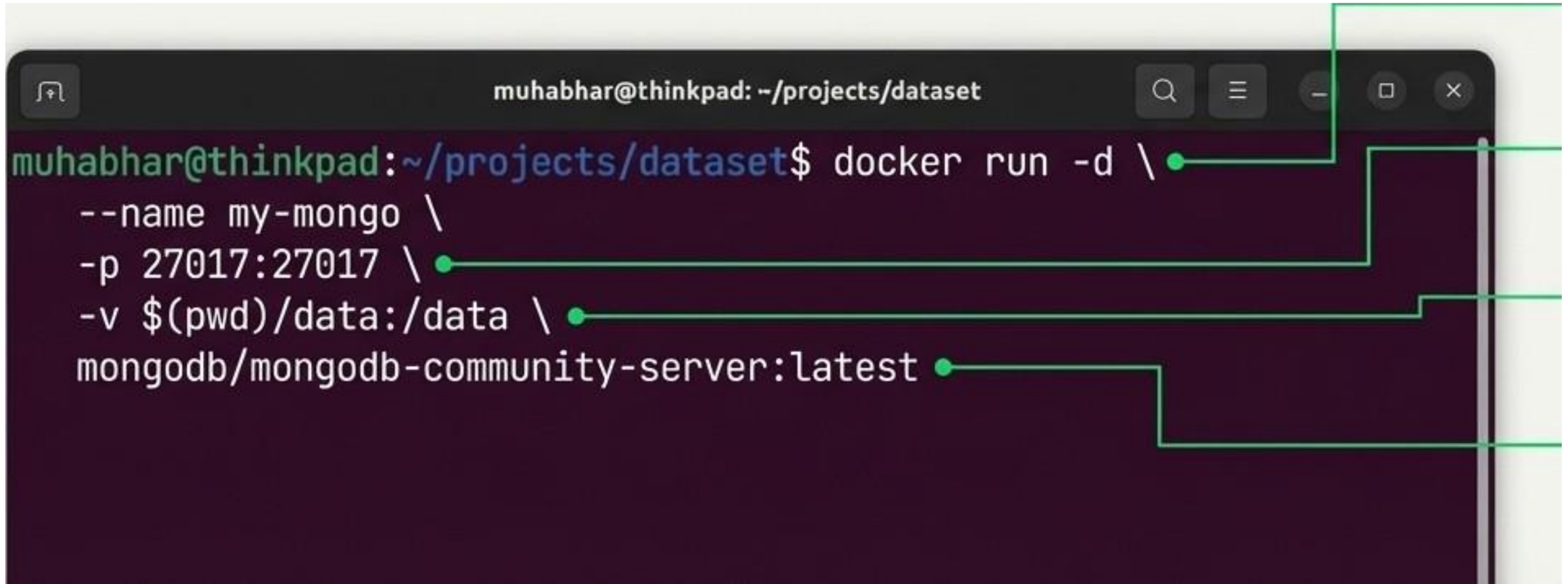
A separate collection to store potentially millions of user reviews. Each document links back to a game via ‘app_id’.

This separation prevents ”games’ documents from becoming excessively large and allows for efficient aggregation and analysis of review data.

The Foundation: Environment Setup with Docker

A containerised approach using Docker was chosen for consistency and reproducibility. This ensures the environmentis isolated and easy to manage.

- Containerisation: Docker Desktop
- Database Server: MongoDB Community Server 8.0 (Official Docker Image)
- CLI Tools: mongoimport (MongoDB command to import the data)
- GUI Client: Web based Mongo Express



`docker run -d`: Run the container in detached mode.

`-p 27017: 27017`: Map the container's port to the host machine.

`-v $(pod)/data: /data`: Mount a local directory for data persistence.

`mongodb/mongodb-community-server:latest` the official docker image.

Making Contact: Connecting to the MongoDB Instance

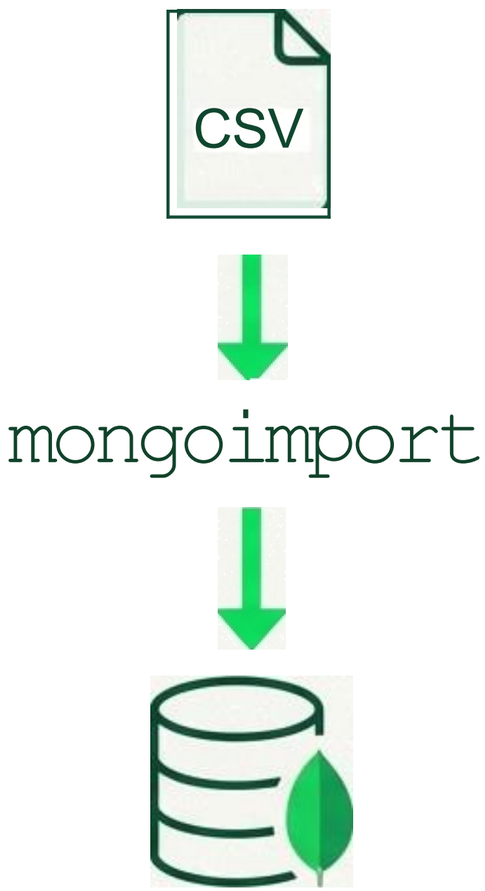
Access the MongoDB shell through the terminal

```
muhabhar@thinkpad: ~/projects/dataset
muhabhar@thinkpad:~/projects/dataset$ docker exec -it my-mongo mongosh \
    -u admin \
    -p pass \
    --authenticationDatabase admin

Current Mongosh Log ID: 69295d9d91234a18569dc29c
Connecting to:          mongodb://<credentials>@127.0.0.1:27017/?directCon
                        nection=true&serverSelectionTimeoutMS=2000&authSou
Using MongoDB:          8.2.2
Using Mongosh:          2.5.5

The server generated these startup warnings when booting:
   2025-11-27T22:11:27.952+00:00: Using the XFS filesystem is strongly re-
commended with the WiredTiger storage engine. See http://dochub.mongodb.o
rg/core/prodnotes-filesystem
   2025-11-27T22:11:28.232+00:00: For customers running the current memory
allocator, we suggest changing the contents of the following sysfsFile
   2025-11-27T22:11:28.232+00:00: For customers running the current memory
allocator, we suggest changing the contents of the following sysfsFile
   2025-11-27T22:11:28.232+00:00: We suggest setting the contents of sysfs
File to 0.
   2025-11-27T22:11:28.233+00:00: vm.max_map_count is too low
   2025-11-27T22:11:28.233+00:00: We suggest setting swappiness to 0 or 1,
as swapping can cause performance problems.
test>
```

Successful Connection established.

The Ingestion: A Bulk Import Strategy

With a 1.5GB dataset, an efficient import strategy was critical. The `mongoimport` utility was selected as the optimal tool for this bulk operation.

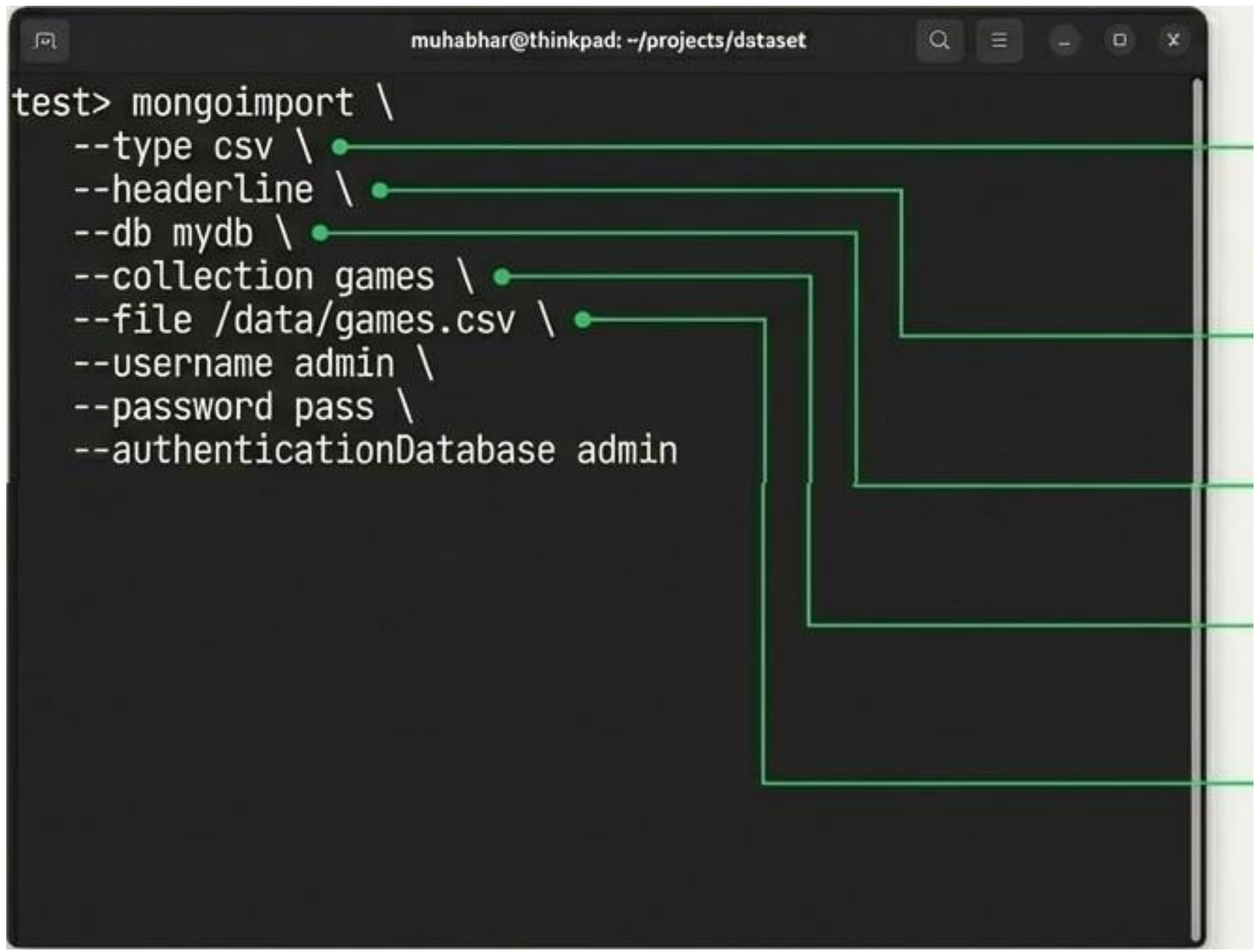


Rationale

- Optimised for Bulk: mongoimport is specifically designed for high-performance ingestion of large files.
- Direct CSV/JSON Support: Natively handles the format of the source data.
- Superior Performance: Significantly faster than scripted `insert8any` operations for initial data loads.

Execution: Importing 85,000+ Game Records

The ‘Mongoimport’ command was executed from the ‘Mongosh’ shell to ingest the ‘games.csv’ file into the ‘games’ collection within the ‘mydb’ database.



File format CSV

First line of the file is
A headlerine for keys

Targets ‘mydb’
database

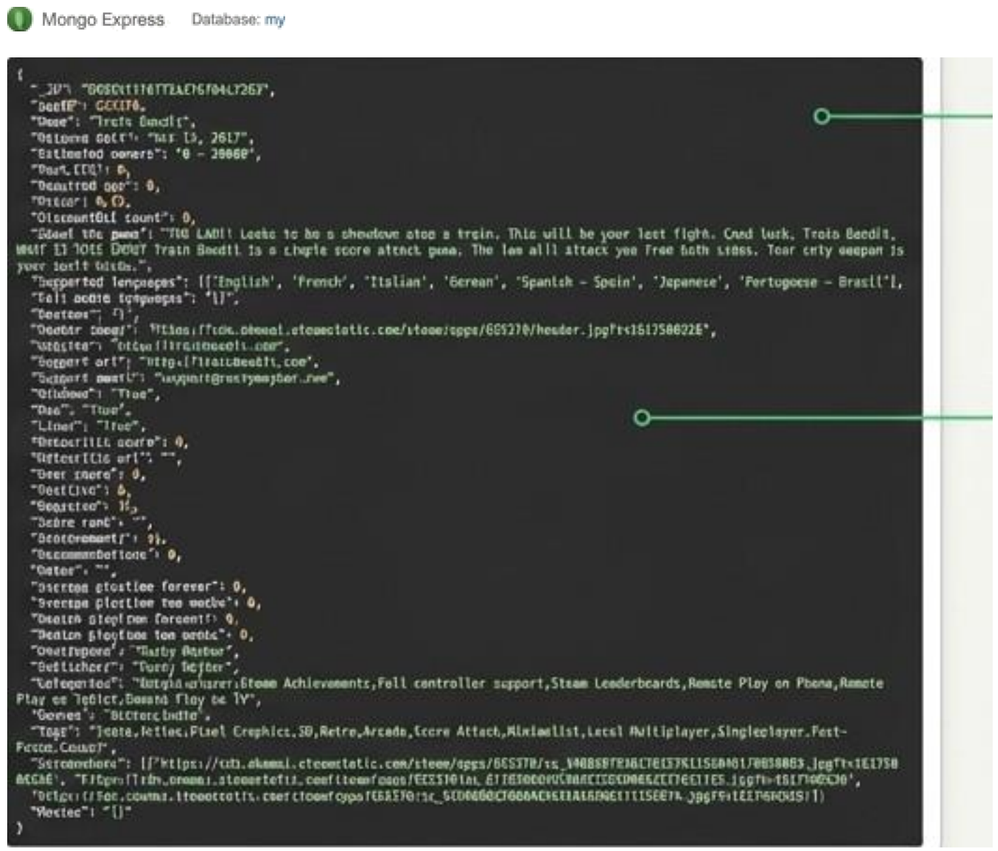
Targets “games”
collection

Path of the csv file
Within the container
Actually (mapped from
The System)

Verification: Confirming the Data Load

Following the import: Mongo Express, a web-based admin UI, was used to visually inspect the database and confirm the data was loaded successfully and structured as expected.

Database Created: The mydb' database is visible in the server list.



Collection Populated: Data is present in the games* collection.

Document Structure: Fields and values are correctly mapped from the CSV source.

The Optimisation: A Post-Import Indexing Strategy

To ensure high-performance queries, indexes were created on the games collection. This crucial step was performed *after* the bulk data import, as building indexes during ingestion significantly slows down the process.

Key Indexes Created

Field	Index Type	Purpose
app_1d	Unique	Primary key lookup
nane	Text	Free—text search
genres	multi—key	Filter by genre
tags	multi—key	Filter by tag
price	Ascending	Price range queries
release date	Descending	Sort by newest releases

The Rationale: Aligning Indexes with Application Queries

Each index was created with a specific application use case in mind, ensuring the database is primed for the platform's key operations.

User Action	Supporting Index
Searching for “Train Bandit” →	`db.games.createIndex({ name: "text" })`
Finding all ‘RPG’ games →	`db.games.createIndex({ genres: 1 })`
Filtering games under £10 →	`db.games.createIndex({ price: 1 })`
Showing the newest releases →	`db.games.createIndex({ release_date: -1 })`
Finding a specific game profile →	`db.games.createIndex({ app_id: 1 }, { unique: true })`

The Payoff: Data Ready for Analysis

With the data loaded and indexed, the database is now ready to serve the application. We can demonstrate its capabilities with sample queries executed directly via `*mongosh*`.

Example 1: Finding a Specific Game

A find() query successfully retrieves the complete document for a game by its AppID .

```
mydb> db.games.find({ AppID: 655370 })
{
  "_id": ObjectId("...192051263568553895371"),
  "AppID": 655370,
  "Name": "Train Bandit",
  "Release date": "Oct 12, 2017",
  ...
}
```

Example 2: Aggregating Data

An 'aggregate() query groups games by their release year and counts them, demonstrating the database's analytical power.

```
mydb> db.games.aggregate([
  { $group: {
    _id: { $year: "$release_date" },
    count: { $sum: 1 }
  }
}, { $sort: { _id: -1 } } ])
[ { _id: null, count: 111452
}, ... ]
```

This represents games with missing 'release_date' values, a known data quality issue handled gracefully by the query.

Conclusion: A Flexible Backend for Complex Data

The project successfully transformed a large, complex, and semi-structured dataset into a performant, query-ready database backend.

Complex Dataset → Strategic Tooling → Query-Ready Application

Key Takeaways

- Embrace Data's Shape: Choosing a database that matches the data's structure (variable, nested) is more effective than forcing the data into a rigid schema.
- Plan the Full Workflow: Success depends on a holistic approach, from environment setup and import strategy to a query-driven indexing plan.
- MongoDB as a Solution: For applications built on real-world, user-generated datasets, MongoDB's flexible document model provides a powerful and scalable foundation.